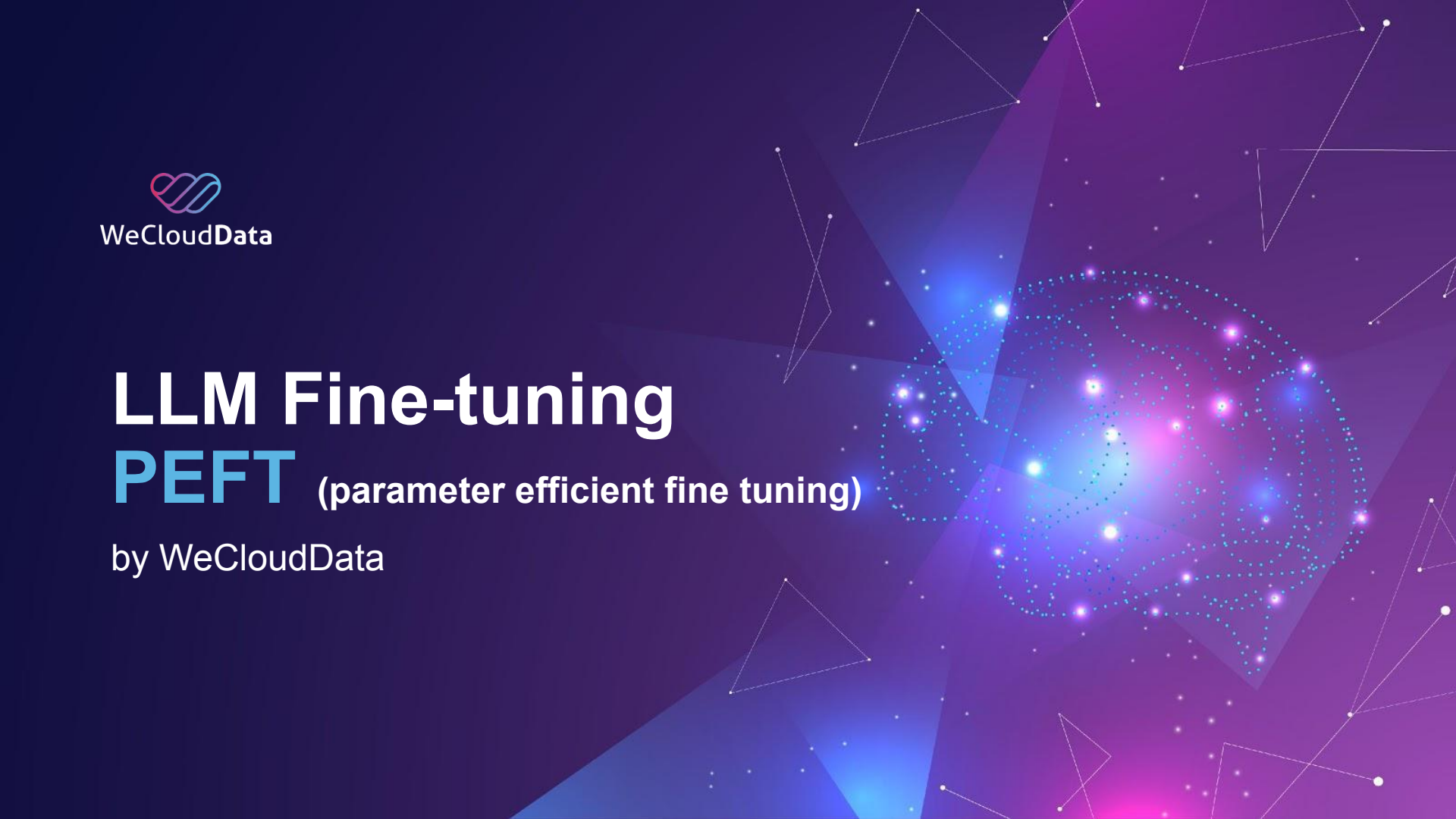


LLM Fine-tuning

PEFT (parameter efficient fine tuning)

by WeCloudData

Using LLMs without Parameter Tuning

Overview

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-Tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- Adapters

PEFT: Low-Rank Adaptation

- LoRA
- QLoRA

Agenda.

In-Context Learning

Using LLM w/o Fine-tuning

In-Context Learning:

GPT-2 (Radford et al.) and GPT-3 (Brown et al.) are generative large language models (LLMs) pretrained on a general text corpus.

- These models are zero/few-shot learners, i.e., we can directly provide a few examples of a target task via the input prompt.
- This is particularly useful if we access the model only through API.

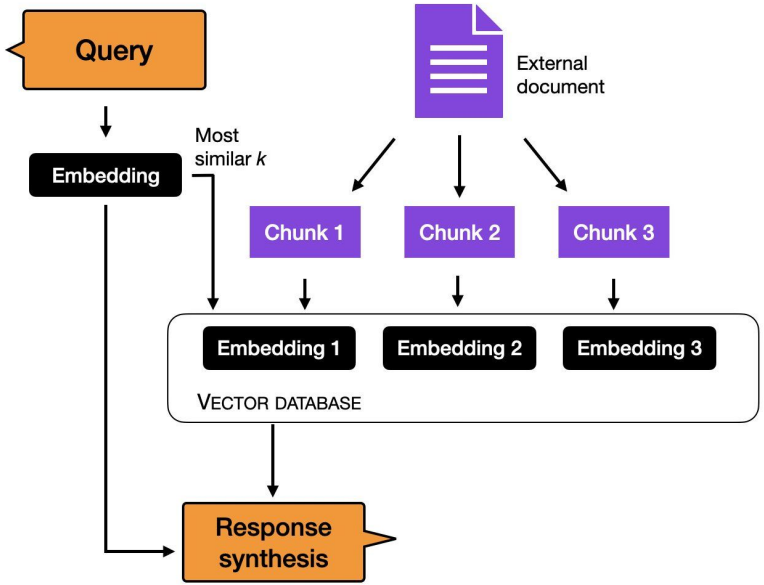
```
Translate the following German sentences into English:

Example 1:
German: "Ich liebe Eis."
English: "I love ice cream."

Example 2:
German: "Draußen ist es stürmisch und regnerisch"
English: "It's stormy and rainy outside."

Translate this sentence:
German: "Wo ist die naechste Supermarkt?"
```

An example of in-context learning.

Reference: Fine-tuning Large Language Models (sebastian)

Retrieval Augmented Generation

Using LLM w/o Fine-tuning



Reference: Fine-tuning Large Language Models (sebastian)

LLM Fine-tuning

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-Tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- Adapters

PEFT: Low-Rank Adaptation

- LoRA
- QLoRA

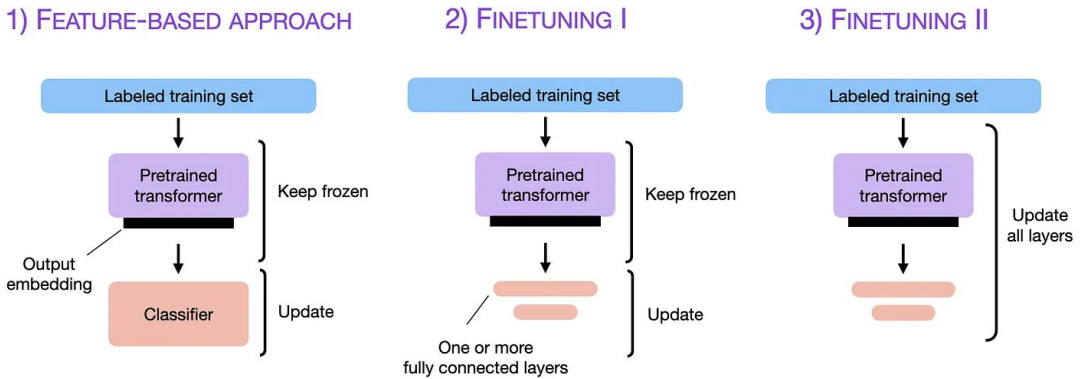
Agenda.

Fine-tuning

While prompting and instruction-tuning can yield the desired results, the subsequent step to enhance performance is to proceed with fine-tuning.

Fine-Tuning:

- We take a pre-trained a Large Language Model and finetune it on a target task.



The 3 conventional feature-based and finetuning approaches.

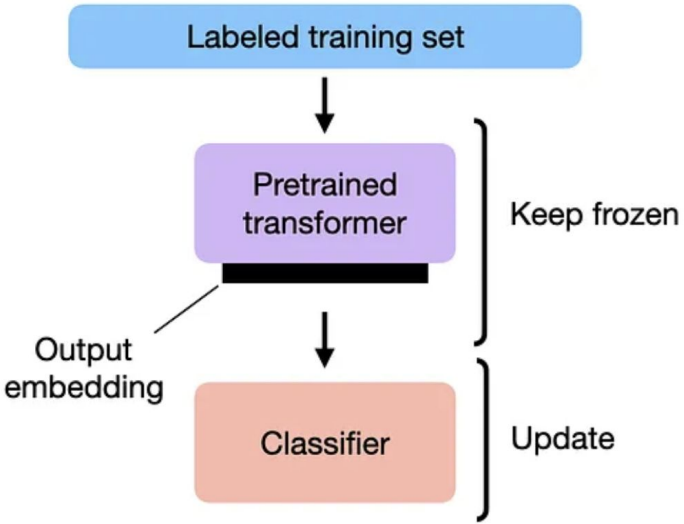
Reference: Fine-tuning Large Language Models (sebastian)

Feature-based Approach

Fine-tuning

Feature-Based Approach:

- Feature-based approach involves loading a pre-trained LLM and applying it to the target dataset.
- Focus on generating output embeddings for the training set, serving as input features for a classification model.
- Commonly used for embedding-focused models like BERT, but applicable to generative GPT-style models as well.
- The classification model can be logistic regression, random forest, XGBoost, etc.



Reference: Fine-tuning Large Language Models (sebastian)

Feature-based Approach

Fine-tuning

```
model = AutoModel.from_pretrained("distilbert-base-uncased")

# ...
# tokenize dataset
# ...

# generate embeddings
@torch.inference_mode()
def get_output_embeddings(batch):
    output = model(
        batch["input_ids"],
        attention_mask=batch["attention_mask"]
    ).last_hidden_state[:, 0]
return {"features": output}

dataset_features = dataset_tokenized.map(
  get_output_embeddings, batched=True, batch_size=10)
```

Generate embeddings

```
X_train = np.array(imdb_features["train"]["features"])
y_train = np.array(imdb_features["train"]["label"])

X_val = np.array(imdb_features["validation"]["features"])
y_val = np.array(imdb_features["validation"]["label"])

X_test = np.array(imdb_features["test"]["features"])
y_test = np.array(imdb_features["test"]["label"])

# train classifier
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X_train, y_train)

print("Training accuracy", clf.score(X_train, y_train))
print("Validation accuracy", clf.score(X_val, y_val))
print("test accuracy", clf.score(X_test, y_test))
```

Classifiers

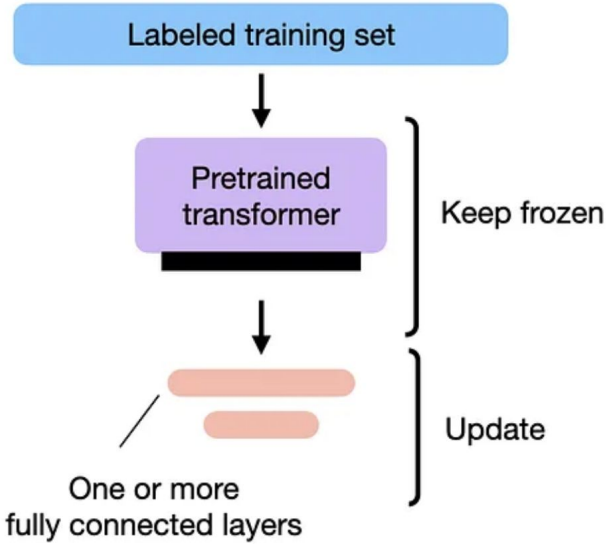
Reference: Fine-tuning Large Language Models (sebastian)

Fine-tuning I

Fine-tuning

Fine-tuning I – Updating The Output Layers:

- Similar to the feature-based approach, parameters of the pre-trained LLM are kept frozen.
- Only newly added output layers are trained.



Reference: Fine-tuning Large Language Models (sebastian)

Fine-tuning I

Fine-tuning

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
     num_labels=2
)
```

Model

```
# freeze all layers
for param in model.parameters():
    param.requires_grad = False
```

Freeze Layers

```
# then unfreeze the two last layers (output layers)
for param in model.pre_classifier.parameters():
    param.requires_grad = True

for param in model.classifier.parameters():
    param.requires_grad = True

# finetune model
lightning_model = CustomLightningModule(model)
```

```
trainer = L.Trainer(
    max_epochs=3,
    ...
)

trainer.fit(
  model=lightning_model,
  train_dataloaders=train_loader,
  val_dataloaders=val_loader)
```

Train

```
# evaluate model
trainer.test(lightning_model, dataloaders=test_loader)
```

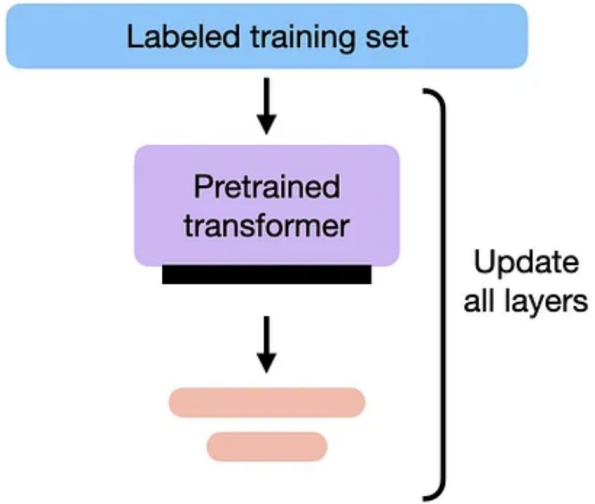
Reference: Fine-tuning Large Language Models (sebastian)

Fine-tuning II

Fine-tuning

Fine-tuning II – Updating All Layers:

- To optimize modeling performance, the preferred approach is fine-tuning all layers (referred to as fine-tuning II).
- Conceptually similar to fine-tuning I, but without freezing the parameters of the pre-trained LLM, fine-tuning them as well.



Reference: Fine-tuning Large Language Models (sebastian)

Fine-tuning II

Fine-tuning

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
     num_labels=2
)
```

Model

```
# freeze layers (which we don't do here)
# for param in model.parameters():
#    param.requires_grad = False
```

Unfreeze Layers

```
# finetune model
lightning_model = LightningModel(model)

trainer = L.Trainer(
    max_epochs=3,
    ...
)

trainer.fit(
  model=lightning_model,
  train_dataloaders=train_loader,
  val_dataloaders=val_loader)

# evaluate model
trainer.test(lightning_model, dataloaders=test_loader)
```

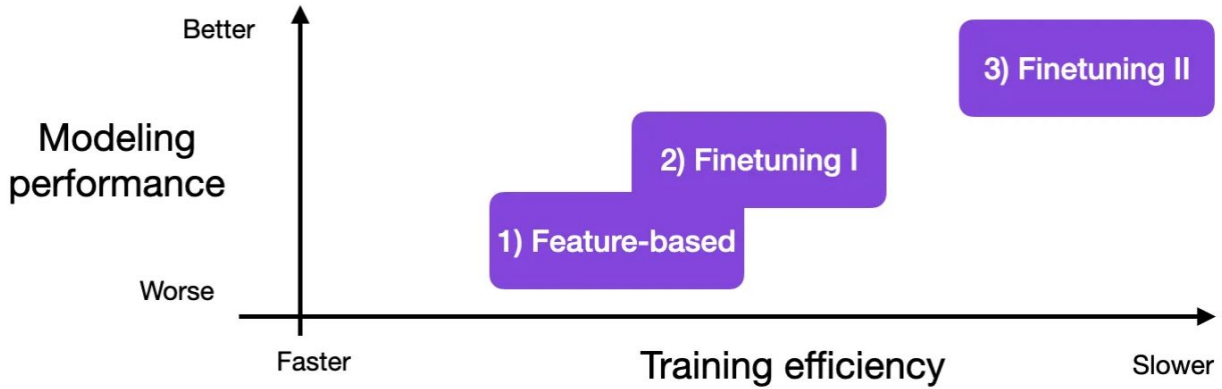
Train

Reference: Fine-tuning Large Language Models (sebastian)

Fine-tuning Methods

Fine-tuning

Ensuring efficiency is crucial because these foundational models are exceptionally large, and it's important to confirm they are compatible with GPU memory constraints.



Rule-of-thumb computational and modeling performance trade-offs for various approaches.

Reference: Fine-tuning Large Language Models (sebastian)

LLM Fine-tuning

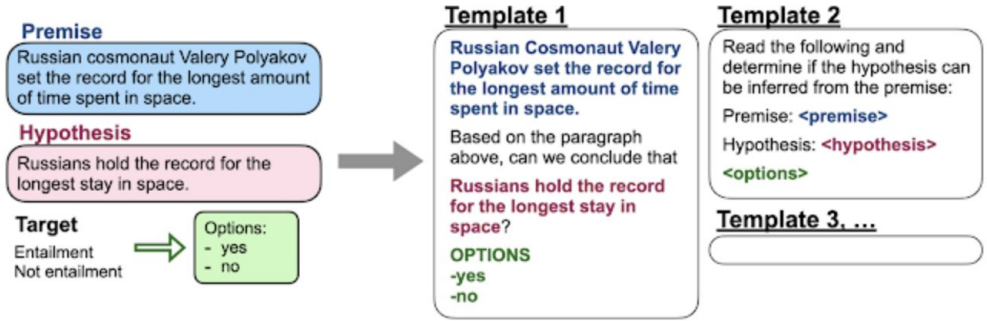
Supervised Instruction Fine-tuning

LLM Fine-tuning

Instruction Tuning

Instruction Tuning:

- Adjusts the model weights through fine-tuning using a diverse set of instructions, employing straightforward and intuitive task descriptions, such as "Classify this movie review as positive or negative" or "Translate this sentence to Danish."



Example templates for a natural language inference dataset.

https://blog.research.google/2021/10/introducing-flan-more-generalizable.html

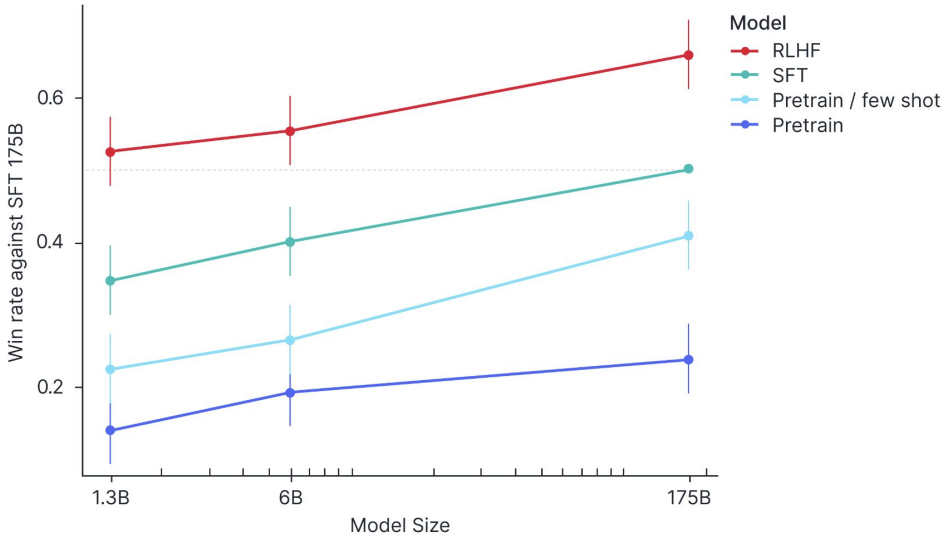
Instruction Tuning

Instruction Tuning

Training language models to follow instructions with human feedback (paper)

Instruction Tuning

Instruction Tuning



Training language models to follow instructions with human feedback (paper)

LLM Fine-tuning

PEFT: Additive Methods

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- Adapters

PEFT: Low-Rank Adaptation

- LoRA
- QLoRA

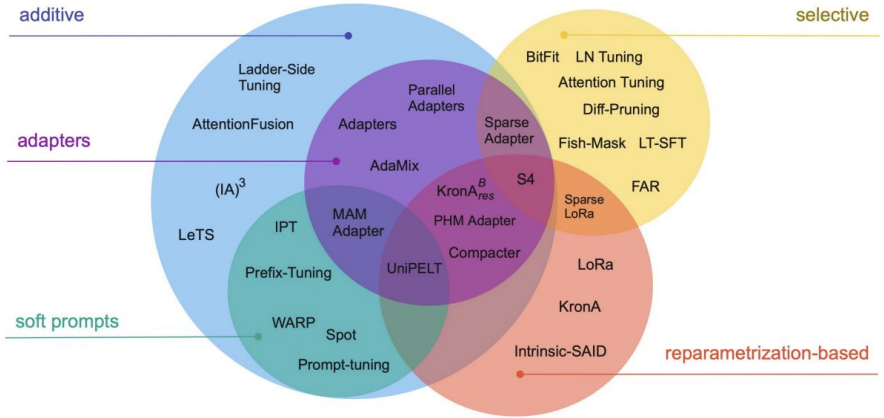
Agenda.

PEFT

PEFT Intro

Parameter-Efficient Fine-Tuning (PEFT)

- Fine-tuning only the last layer is not enough to obtain a good performance
- We need to fine-tune all the layers, but it is too expensive.
- The goal is to fine-tune all the layers in an efficient way, i.e., Parameter-Efficient Fine-Tuning (PEFT).



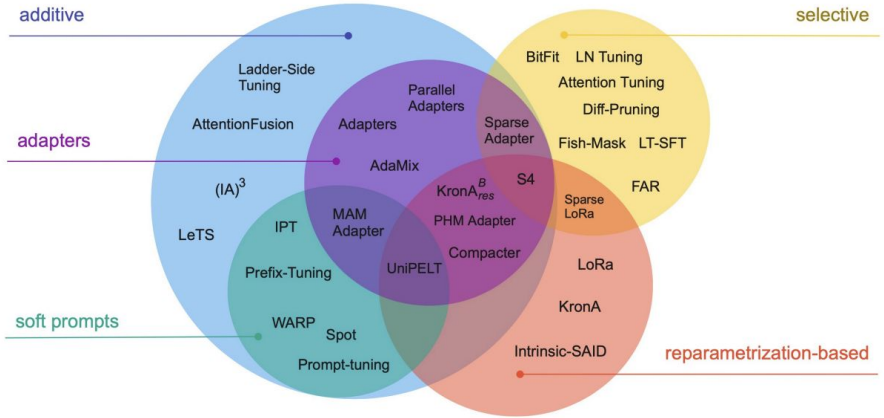
Reference: Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

PEFT

PEFT Intro

Parameter- Efficient Fine-Tuning (PEFT)

- PEFT involves preserving the weights of the original LLM while training a limited number of task-specific adapter layers and parameters.
- PEFT exhibits increased resilience against catastrophic forgetting, given that the majority of pre-trained weights remain unchanged.



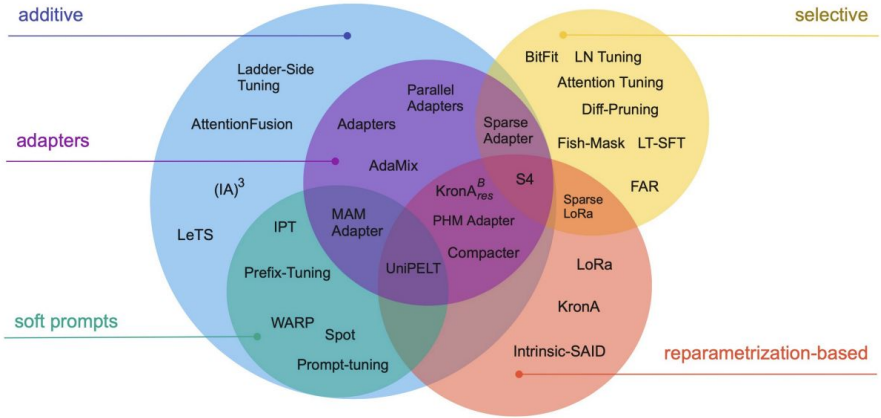
Reference: Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

PEFT

PEFT Intro

Parameter- Efficient Fine-Tuning (PEFT)

- In PEFT, most, if not all, LLM weights remain frozen.
- The number of trained parameters is significantly smaller than the original LLM. It can be as smaller as 0.1% of the original size.
- This reduction makes memory requirements for training more manageable.
- PEFT can often be executed on a single GPU due to its reduced parameter set.
- The original LLM is only slightly modified or left unchanged in PEFT.
- PEFT is less susceptible to the catastrophic forgetting issues associated with full fine-tuning.



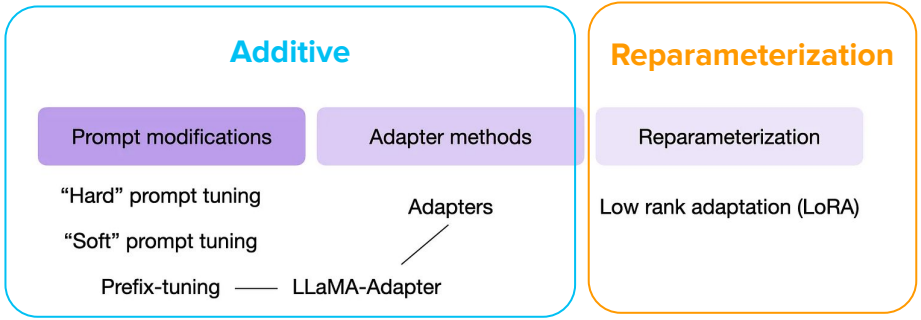
Reference: Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

Different Types of PEFT Methods

PEFT Intro

In the Scaling Down to Scale Up paper, the authors created a taxonomy that categorizes PEFT into the following categories

- **Additive methods**
- Selective methods
- **Low-Rank Adaptation-based methods**
- Hybrid methods



Reference: Understanding Parameter-Efficient LLM Fine-tuning: Prompt Tuning And Prefix Tuning (sebastian)

Different Types of PEFT Methods

PEFT Intro

- **Additive methods**
  - The main idea behind additive methods is augmenting the existing pre-trained model with extra parameters or layers and training only the newly added parameters.
- Selective methods
- Low-Rank Adaptation-based methods
- Hybrid methods

Adapters

Adapter methods add new trainable layers to the architecture of the model, typically inside the encoder or decoder components after the attention or feed-forward layers.

Soft Prompts

Soft Prompting methods keep the model architecture fixed and frozen, and focus on manipulating the input to achieve better performance. This can be done by adding trainable parameters to the prompt embeddings or keeping the input fixed and retraining the embedding weights.

LLaMA Adapter

Prefix Tuning

Prompt Tuning

Different Types of PEFT Methods

PEFT Intro

- Additive methods
- Selective methods
- **Low-Rank Adaptation-based methods**
  - These methods use the idea of reparametrizing the weights of the network using a low-rank transformation. This decreases the trainable parameter count while still allowing the method to work with high-dimensional matrices, such as the pre-trained parameters of the networks.
- Hybrid methods

LoRA

Low-Rank Adaptation provides the modular approach towards to fine-tuning a model for domains specific tasks and provides the capability of transfer learning. LoRA technique can be implemented with fewer resources and are memory efficient.

QLoRA

QLoRA extends LoRA to enhance efficiency by quantizing weight values of the original network, from high-resolution data types, such as Float32, to lower-resolution data types like int4. This leads to reduced memory demands and faster calculations.

What matters in PEFT?

PEFT Intro

- Number of parameters… which ones?
  - Trainable — memory
  - Changed — storage
  - Rank of the delta — how much the network subspace is affected
- Training efficiency
  - Do we add parameters to the network? How expensive are they?
  - Does the method require to backpropagate through the original network?
  - Can the method efficiently utilize the GPU?
- Inference efficiency
  - Do we add parameters to the network? How expensive are they?
- Accuracy
  - Do we get good performance out of the network in the end?

Reference: Understanding Parameter-Efficient LLM Fine-tuning: Prompt Tuning And Prefix Tuning (sebastian)

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-tuning

Instruction Tuning

PEFT: Additive Methods

- **Soft Prompting**
- Adapters

PEFT: Low-Rank Adaptation

- LoRA
- QLoRA

Agenda.

Hard Prompt Tuning

Soft Prompts: Prompt Tuning

Hard Prompt Tuning: The initial notion of prompt tuning involves employing techniques that manipulate the input prompt to enhance modeling outcomes. For instance, consider a scenario where we aim to translate an English sentence into German, and we can ask in various different ways:

```
1) "Translate the English sentence '{english_sentence}' into German: {german_translation}"

2) "English: '{english_sentence}' | German: {german_translation}"

3) "From English to German: '{english_sentence}' -> {german_translation}"
```

An example of hard prompt tuning, that is, rearranging the input to get better outputs.

Reference: Understanding Parameter-Efficient LLM Fine-tuning: Prompt Tuning And Prefix Tuning (sebastian)

Soft Prompt Tuning

Soft Prompts: Prompt Tuning

Soft Prompt Tuning: Differing from the approach of hard prompt tuning, soft prompt tuning (Lester et al. 2021) involves the concatenation of input token embeddings with a trainable tensor. This tensor can be optimized through backpropagation to enhance modeling performance in a specific target task.

```
soft_prompt = torch.nn.Parameter( # Make tensor trainable
    torch.rand(num_tokens, embed_dim)) # Initialize soft prompt tensor

def input_with_soft_prompt(x, soft_prompt) :
    x = concatenate([soft_prompt, x], # Prepend soft prompt to input
                    dim=seq_len)
    return x

# train soft prompt tensor via gradient descent
train(model(input_with_soft_prompt(x)))

# use model with soft prompts
model(input_with_soft_prompt(x))
```

Pseudo-code illustrating the concept of soft prompting.

Reference: Understanding Parameter-Efficient LLM Fine-tuning: Prompt Tuning And Prefix Tuning (sebastian)

LLM Fine-tuning

Soft Prompts: Prompt Tuning

Soft prompt → Fine-tune part of your attention input

Pseudocode

```
def soft_prompted_model(input_ids):
    x = Embed(input_ids)
    x = concat([soft_prompt, x], dim=seq)
    return model(x)
```

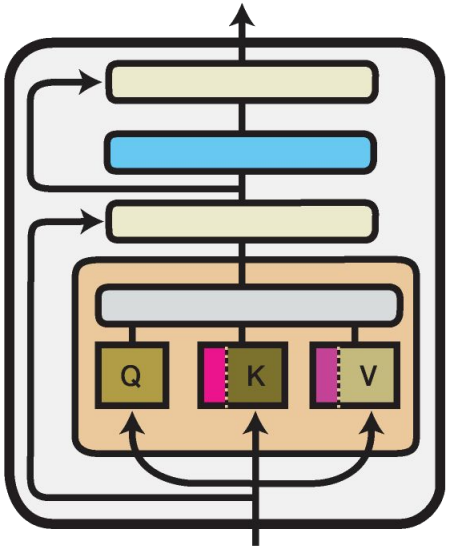


Image Source: Adapter-Transformers v3 - Unifying Efficient Fine-Tuning (link)

Reference:

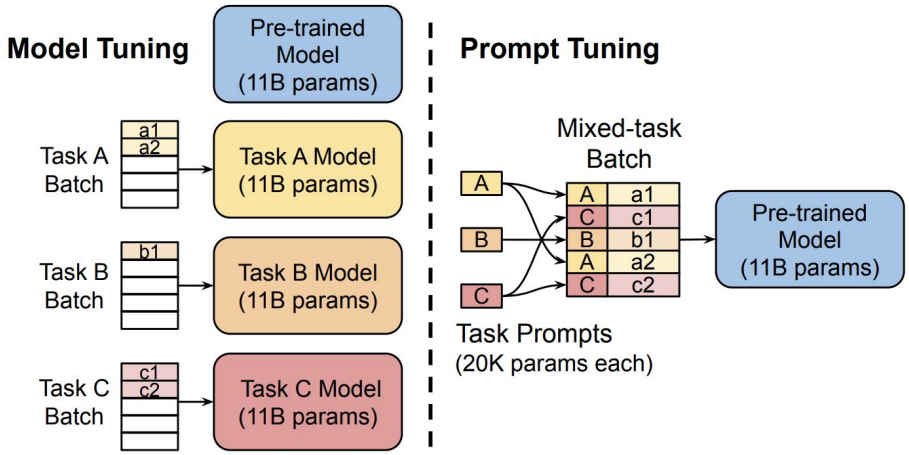
- The Power of Scale for Parameter-Efficient Prompt Tuning (paper)
- Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

Prompt Tuning

Soft Prompts: Prompt Tuning

Prompt-Tuning

- The weights of the model are frozen.
- The embedding vectors for the prompt tokens gets updated during the training.
- In contrast to updating Millions to Billions of parameters in full-fine tuning, only 10K-100K parameters get updated.
- Similar to LORA you can train a set of different soft prompts for different tasks and switch them during the inference time.



- **Model tuning** requires making a task-specific copy of the entire pre-trained model for each downstream task and inference must be performed in separate batches.
- **Prompt tuning** only requires storing a small task-specific prompt for each task, and enables mixed-task inference using the original pretrained model.

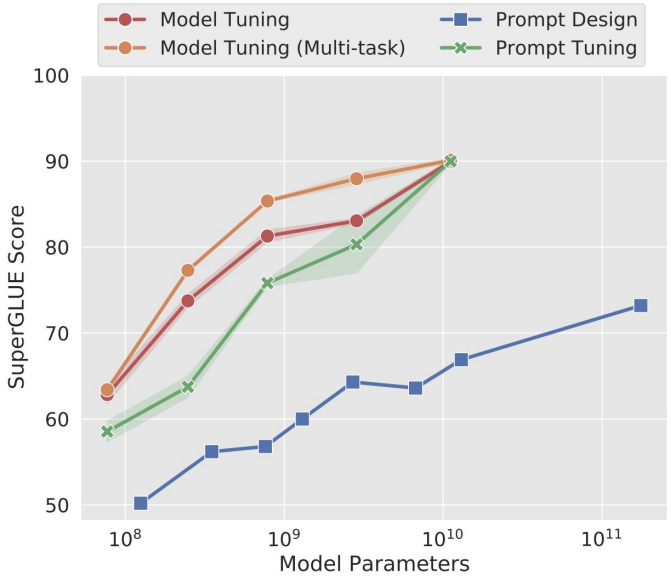
Reference: The Power of Scale for Parameter-Efficient Prompt Tuning (paper)

Prompt Tuning

Soft Prompts: Prompt Tuning

Prompt-Tuning

- Prompt tuning initially lags behind full fine tuning for smaller LLMs. However, as the model size increases, prompt tuning performance improves.
- Models with around 10 billion parameters see prompt tuning as effective as full fine tuning, offering a significant boost in performance over prompt engineering alone.



Standard model tuning of T5 achieves strong performance, but requires storing separate copies of the model for each end task. The prompt tuning of T5 matches the quality of model tuning as size increases, while enabling the reuse of a single frozen model for all tasks. The author's approach significantly outperforms fewshot prompt design using GPT-3.

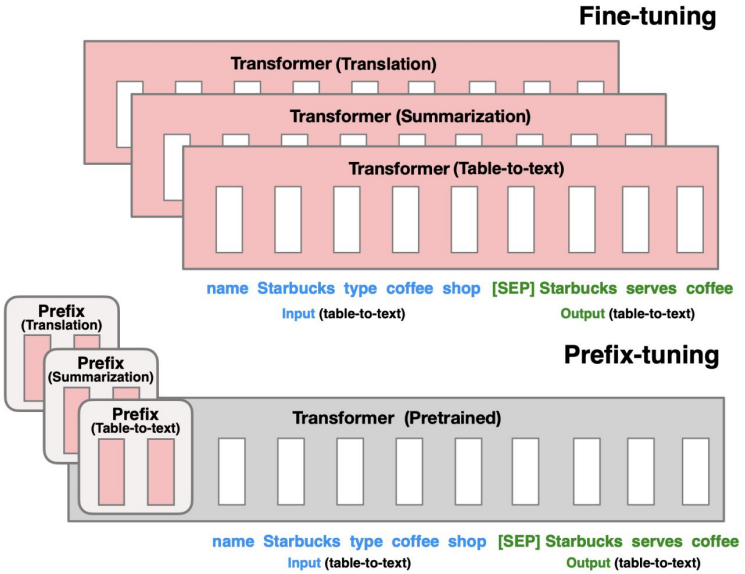
Reference: The Power of Scale for Parameter-Efficient Prompt Tuning (paper)

Prefix-tuning

Soft Prompts: Prefix Tuning

Prefix tuning: add trainable tensors *to each* transformer block instead of only the input embeddings, as in *soft* prompt tuning.

```
def transformer_block_for_prefix_tuning(x):
    soft_prompt = FFN(soft_prompt)
    x = concat([soft_prompt, x], dim=seq)
    return transformer_block(x)
```



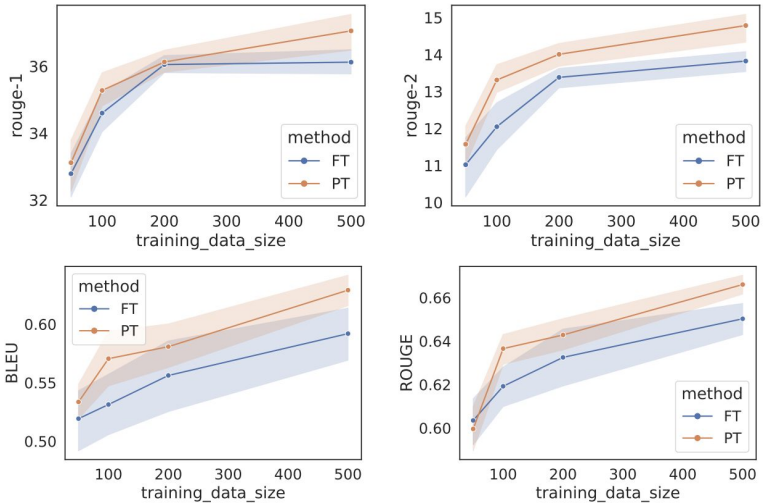
- **Fine-tuning** (top) updates all Transformer parameters (the red Transformer box) and requires storing a full model copy for each task.
- The authors propose **prefix-tuning** (bottom), which freezes the Transformer parameters and only optimizes the prefix (the red prefix blocks). Consequently, we only need to store the prefix for each task, making prefix-tuning modular and space-efficient. Note that each vertical block denote transformer activations at one time step.

Reference: Prefix-Tuning: Optimizing Continuous Prompts for Generation (paper)

Prefix-tuning

Soft Prompts: Prefix Tuning

Prefix tuning: add trainable tensors *to each* transformer block instead of only the input embeddings, as in *soft* prompt tuning.



Prefix-tuning (orange) outperforms **fine-tuning** (blue) in low-data regimes in addition to requiring many fewer parameters. The top two plots correspond to summarization, measured by ROUGE-1 and ROUGE-2. The bottom two plots correspond to table-to-text, measured by BLEU and ROUGE-L. The x-axis is the training size and the y-axis is the evaluation metric (higher is better).

Reference: Prefix-Tuning: Optimizing Continuous Prompts for Generation (paper)

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- **Adapters**

PEFT: Low-Rank Adaptation

- LoRA
- QLoRA

Agenda.

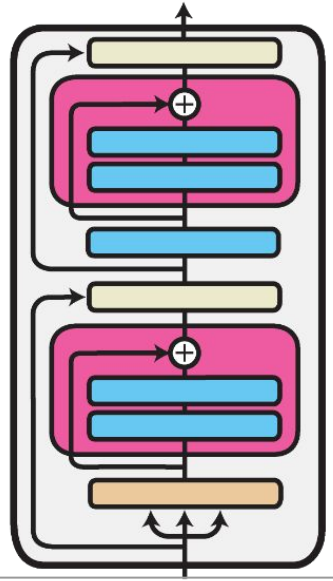
Adapters

Additive Method: Adapters

Adapters: Add fully-connected networks after attention and FFN layers

Pseudocode

```
def transformer_block_with_adapter(x):
    residual = x
    x = SelfAttention(x)
    x = FFN(x)   # adapter
    x = LN(x + residual)
    residual = x
    x = FFN(x)   # transformer FFN
    x = FFN(x)   # adapter
    x = LN(x + residual)
    return x
```



Unlike the transformer FFN block, Adapters usually have a smaller hidden dimension than the input. Adapters have demonstrated impressive parameter efficiency at the time, showing that it is possible to achieve performance competitive to full fine-tuning by tuning less than 4% of the total model parameters.

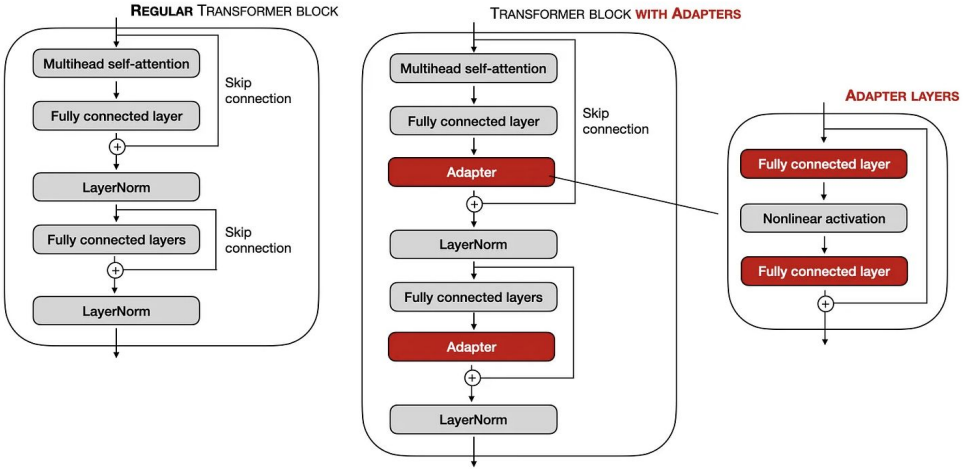
Reference:

- The Power of Scale for Parameter-Efficient Prompt Tuning (paper)
- Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning (paper)

Adapters

Additive Method: Adapters

- Unlike the prefix tuning, where tunable tensors are prepended to the embeddings, in the adapter method, adapter layers are added in two specific locations within the model.
- According to the original adapter paper, utilizing the adapter method in training a BERT model achieves modeling performance similar to a fully fine-tuned BERT model with the need to train only 3.6% of the parameters.



Comparison of the regular transformer blocks used in various LLMs and a transformer block modified via the adapter layers.

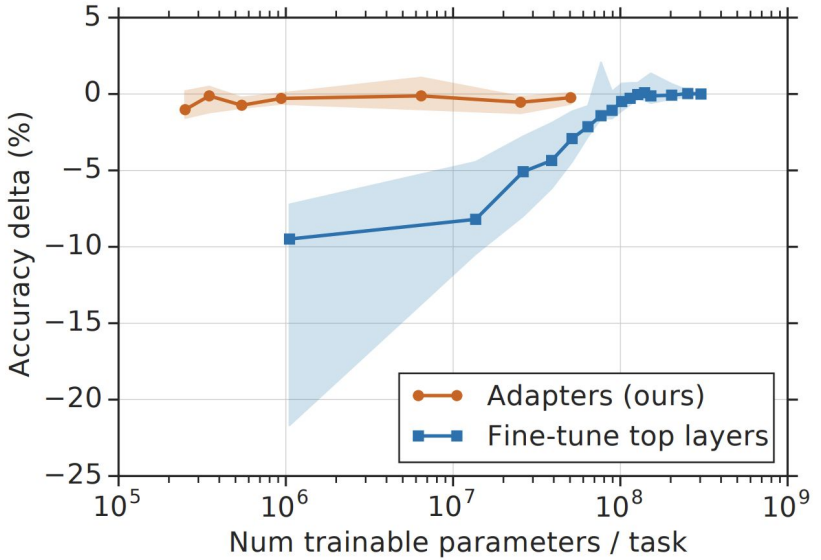
Reference: Fine-tuning LLMs Efficiently with Adapters (sebastian)

Adapters

Additive Method: Adapters

In the plot, the authors shows the trade-off between accuracy and number of trained task-specific parameters, for adapter tuning and fine-tuning.

- The y-axis is normalized by the performance of full fine-tuning.
- The curves show the 20th, 50th, and 80th performance percentiles across nine tasks from the GLUE benchmark.
- Adapter-based tuning attains a similar performance to full fine-tuning with two orders of magnitude fewer trained parameters

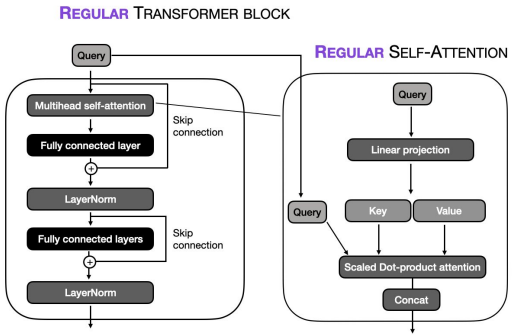


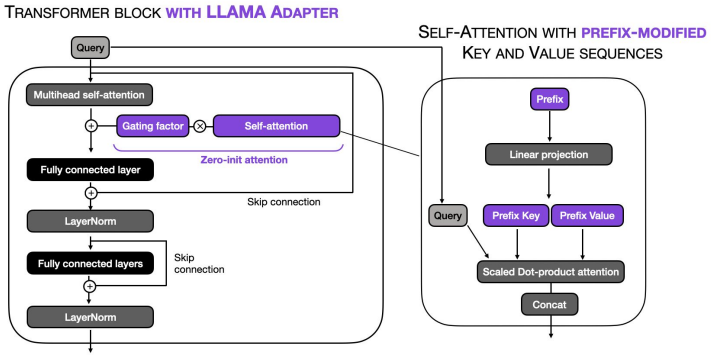
Reference: Parameter-Efficient Transfer Learning for NLP (paper)

LLaMA-Adapters

Additive Method: Adapters

- Like prefix tuning, the LLaMA-Adapter method prepends tunable prompt tensors to the embedded inputs. The prefix is learned and maintained within an embedding table rather than being provided externally. Each transformer block in the model has its own distinct learned prefix, allowing for more tailored adaptation across different model layers.
- In addition, LLaMA-Adapter introduces a ***zero-initialized attention mechanism*** coupled with gating.





Reference:

- Understanding Parameter-Efficient Fine-tuning of Large Language Models: From Prefix Tuning to LLaMA-Adapters (paper)

LLM Fine-tuning

PEFT: Low-Rank Adaptation

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- Adapters

PEFT: Low-Rank Adaptation

- **LoRA**
- QLoRA

Agenda.

Low-Rank Adaptation

LoRA

Motivation

LoRA lets you build small adapters that you can pair with models and get customized results. These adapters can be fine-tuned for specific datasets or specific users. These adapters are very small, 6MB-8MB, and you only need to apply these adapters to the large model, which is much faster to do in a production environment.

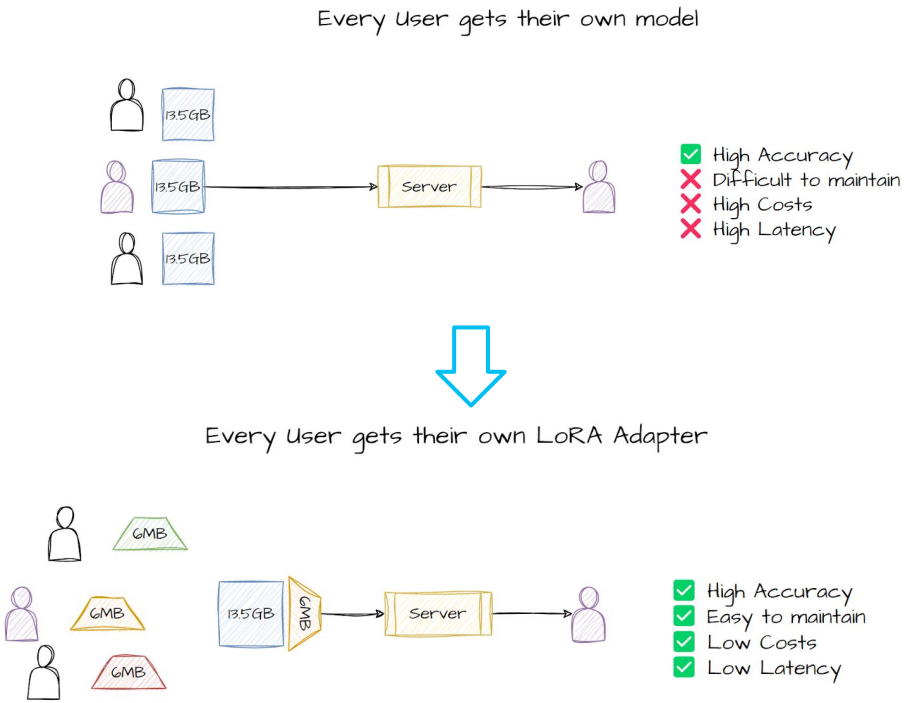


Image Source: In-depth guide to fine-tuning LLMs with LoRA and QLoRA (blog)

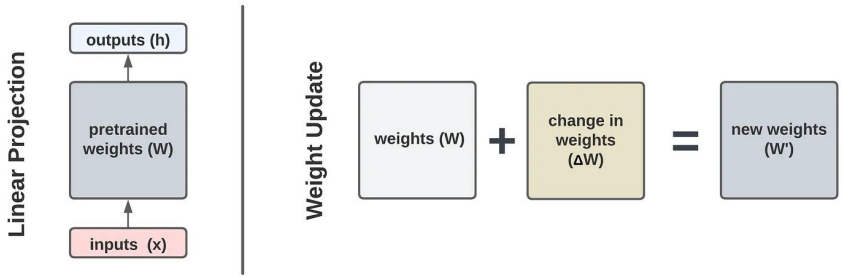
How Does LoRA Work?

LoRA

Traditional Fine-tuning Process

- **Forward Pass**: Compute some set of predictions by passing input data forward through the network.
- **Loss**: Compare the predictions against the ground truth to obtain a measure of loss or error.
- **Backward Pass**: Compute the gradient of the loss with respect to the weights and biases by working backward through the network.
- **Update Weights**: Nudge the weights and biases in the opposite direction of the gradient to reduce the loss.

$$W' = W + \Delta W$$



Reference: Practical Tips for Fine-tuning LLMs (blog)

Image Source: LoRA: Low-Rank Adaptation from Scratch (link)

How Does LoRA Work?

LoRA

Understanding LoRA

Unlike traditional fine-tuning, LoRA keeps the weight matrix and the change-in-weights matrix separate throughout the fine-tuning process.

- This keeps the model size completely the same and still offers the flexibility of parameter-efficient fine-tuning.

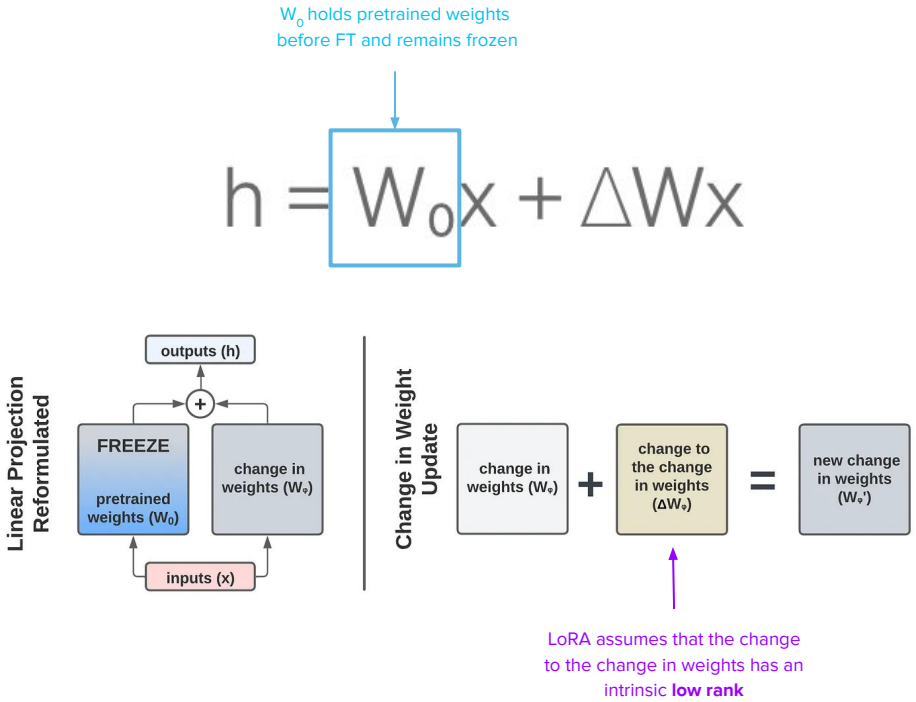


Image Source: LoRA: Low-Rank Adaptation from Scratch (link)

How Does LoRA Work?

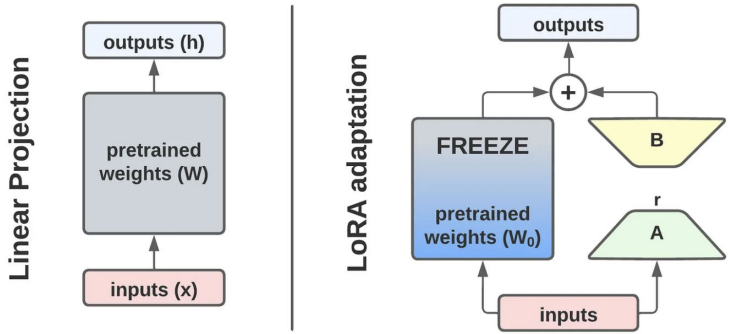
LoRA

The rank of a matrix

The rank of a matrix is the maximum number of linearly independent rows or columns in that matrix. **Low intrinsic rank** refers to the idea that the information contained within the matrix can be represented using fewer dimensions than the original matrix might suggest.

W_φ can be approximated by B x A

$$h = W_0x + W_\varphi x = W_0x + BAx$$



LoRA adaptation of a linear layer.

Reference: Practical Tips for Fine-tuning LLMs (blog)

Image Source: LoRA: Low-Rank Adaptation from Scratch (link)

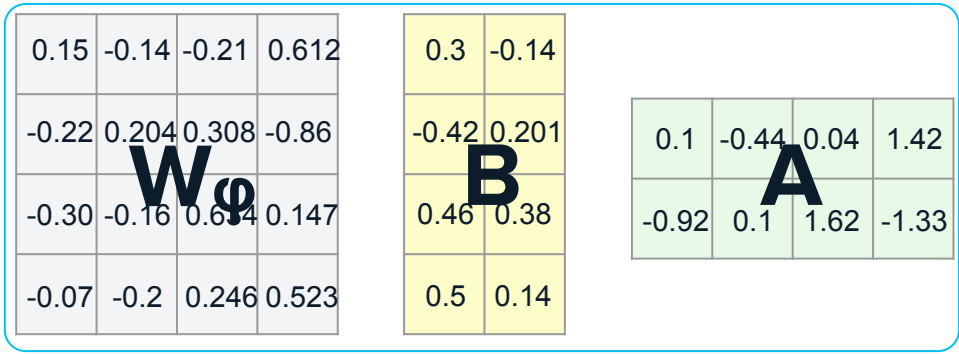
How Does LoRA Work?

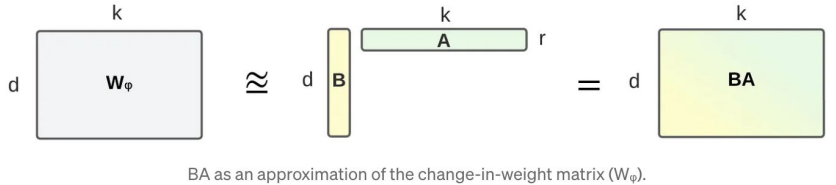
LoRA

Understanding LoRA

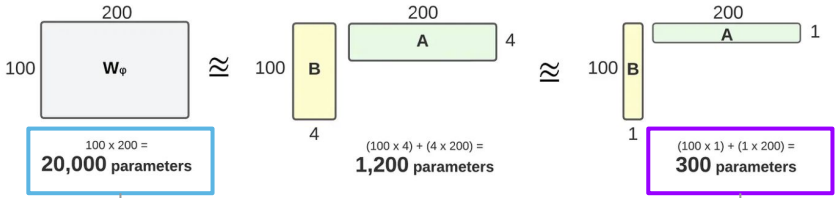
The dimension r is a hyperparameter that we define so that BA has fewer trainable parameters than W_φ. The smaller the value of r the more compressed the representation.

- Matrix decomposition is quite expensive operation. But since B and A are parameterized, they are learned during the fine-tuning process





BA as an approximation of the change-in-weight matrix (W_φ).



Comparison of rank-4 and rank-1 adaptation. LoRA results in fewer trainable parameters.

LoRA results in much fewer trainable parameters

Image Source: LoRA: Low-Rank Adaptation from Scratch (link)

Reference: Practical Tips for Fine-tuning LLMs (blog)

Low-Rank Adaptation

LoRA

- Applying LoRA to the self-attention layers of the model frequently results in performance improvements.
- In principle, LoRA can also be applied to other components, such as the feed-forward layers.
- The attention layers contain the majority of parameters in LLMs.
- The most significant reduction in trainable parameters occurs by applying LoRA to the weight matrices of these attention layers.

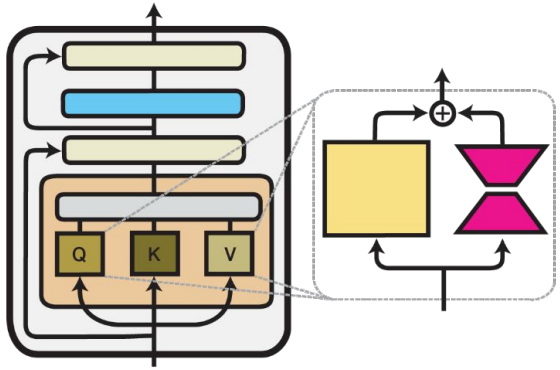


Image Source: adapterhub.ml

Fine-tuning Using LoRA

LoRA

LoRA Training

- Transformer weights dimensions: d x k = 512 x 64.
- We have 512 x 64 = 32768 trainable parameters.
- LoRA with r = 8:
  - A has r x k = 8 x 64 = 512 trainable parameters.
  - B has d x r = 512 x 8 = 4096 trainable parameters.
  - Leads to 86% reduction in trainable parameters.

LoRA during inference time

- Store the rank decomposition matrices for different tasks, and add them to the original weights.

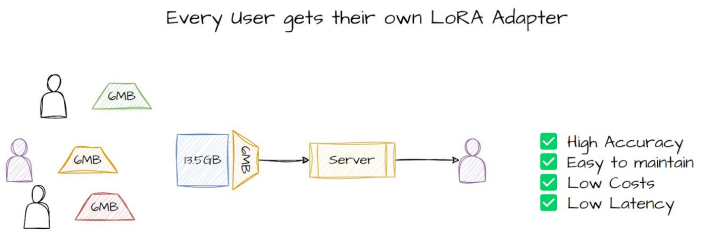


Image Source: In-depth guide to fine-tuning LLMs with LoRA and QLoRA (blog)

Fine-tuning Using LoRA

LoRA

How to choose rank r?

- The authors found a plateau in the loss value for ranks greater than 16.
- The ranks in the range of 4-32 can provide a good trade-off between reducing trainable parameters and preserving performance.
- Optimizing the choice of rank is an ongoing area of research.

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	**70.38**	**8.8439**	**0.4689**	0.7186	**2.5349**
8	1.17	69.57	8.7457	0.4636	**0.7196**	2.5196
16	**1.16**	69.61	8.7483	0.4629	0.7177	2.4985
32	**1.16**	69.33	8.7736	0.4642	0.7105	2.5255
64	**1.16**	69.24	8.7174	0.4651	0.7180	2.5070
128	**1.16**	68.73	8.6718	0.4628	0.7127	2.5030
256	**1.16**	68.92	8.6982	0.4629	0.7128	2.5012
512	**1.16**	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

Reference: LoRA: Low-Rank Adaptation of Large Language Models (paper)

How Does LoRA Work

LoRA

Understanding LoRA

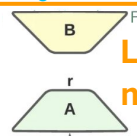
Unlike traditional fine-tuning, LoRA keeps the weight matrix and the change-in-weights matrix separate throughout the fine-tuning process.

Reference: Practical Tips for Fine-tuning LLMs (blog)

```python
class LinearLoRA(nn.Module):
    """
    A low-rank adapted linear layer.

    Args:
        in_dim: int = An integer representing the input dimension of the linear layer
        out_dim: int = An integer representing the output dimension of the linear layer
        r: int = An integer representing the rank of the low-rank approximated matrices
        lora_alpha: int = An integer representing the numerator of the scaling constant alpha / r
        lora_dropout: float = A float between 0 and 1 representing the dropout probability
    """

    def __init__(
        self,
        in_dim: int,
        out_dim: int,
        r: int = 8,
        lora_alpha: int = 16,
        lora_dropout: float = 0.0,
    ):
        super().__init__()
        self.r = r
        self.lora_alpha = lora_alpha
        self.lora_dropout = nn.Dropout(lora_dropout)

        # Check that the rank is at least 1
        assert (
            r > 0
        ), "Variable 'r' is not greater than zero. Choose a rank of 1 or greater."

        # recreate the linear layer and freeze it (the actual weight values will be copied in outside of this class)
        self.pretrained = nn.Linear(in_dim, out_dim, bias=True)
        self.pretrained.weight.requires_grad = False

        # create the low-rank A matrix and initialize with same method
        self.lora_A = nn.Linear(in_dim, r, bias=False)
        nn.init.kaiming_uniform_(self.lora_A.weight, a=math.sqrt(5))

        # create the low-rank B matrix and initialize to zero
        self.lora_B = nn.Linear(r, out_dim, bias=False)
        nn.init.constant_(self.lora_B.weight, 0)

        # scaling constant
        self.scaling = self.lora_alpha / self.r

    def forward(self, x):
        pretrained_out = self.pretrained(x)

        lora_out = self.lora_dropout(x)
        lora_out = self.lora_A(lora_out)
        lora_out = self.lora_B(lora_out)
        lora_out = lora_out * self.scaling
```

r determines the maximum possible rank of the approximated matrices

Freeze W_0



Face PEFT library

Low-rank matrix A, B

How Does LoRA Work?

LoRA

LoRA with Huggingface

To implement LoRA fine-tuning with HuggingFace, you need to use the PEFT library to inject the LoRA adapters into the model and use them as the update matrices.

Once this is done, you can train the model as you would normally do. But this time it will take much less time and compute resources as it normally does.

```python
from transformers import AutoModelForCausalLM
from peft import get_peft_config, get_peft_model, LoraConfig, TaskType

model = AutoModelForCausalLM.from_pretrained(
    model_name_or_path, device_map="auto", trust_remote_code=True
)  # load the model

peft_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    inference_mode=False,
    r=32,
    lora_alpha=16,
    lora_dropout=0.1,
    target_modules=[
        "query_key_value"
    ],  # optional, you can target specific layers using this
)  # create LoRA config for the finetuning

model = get_peft_model(model, peft_config)  # create a model ready for LoRA finetuning

model.print_trainable_parameters()
# trainable params: 9,437,184 || all params: 6,931,162,432 || trainable%: 0.13615586263611604
```

LoRA Configuration

Reference: Practical Tips for Fine-tuning LLMs (blog)

Fine-tuning Falcon 7b with LoRA

LoRA

Finetuned model →

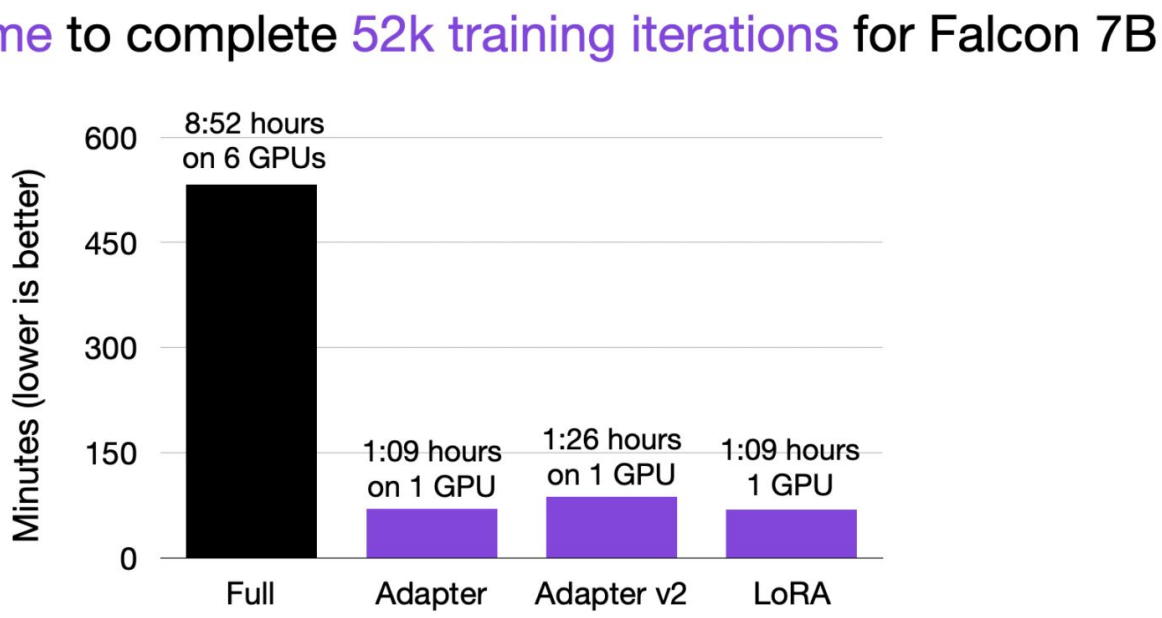
Pretrained foundation (base) model →

Model	Revision	Average	ARC (25-shot)	HellaSwag (10-shot)	MMLU (5-shot)
tiiuae/falcon-40b-instruct	main	63.2	61.6	84.4	54.1
CalderaAI/30B-Lazarus	main	60.7	57.6	81.7	45.2
tiiuae/falcon-40b	main	60.4	61.9	85.3	52.7
timdettmers/guanaco-33b-merged	main	60	58.2	83.5	48.5
ausboss/llama-30b-supercot	main	59.8	58.5	82.9	44.3
huggyllama/llama-65b	main	58.3	57.8	84.2	48.8
llama-65b	main	58.3	57.8	84.2	48.8
MetaIX/GPT4-X-Alpasta-30b	main	57.9	56.7	81.4	43.6
Aeala/VicUnlocked-alpaca-30b	main	57.6	55	80.8	44

Reference: Fine-tuning Falcon LLMs More Efficiently With LoRA and Adapters (link)

Fine-tuning Falcon 7b with LoRA

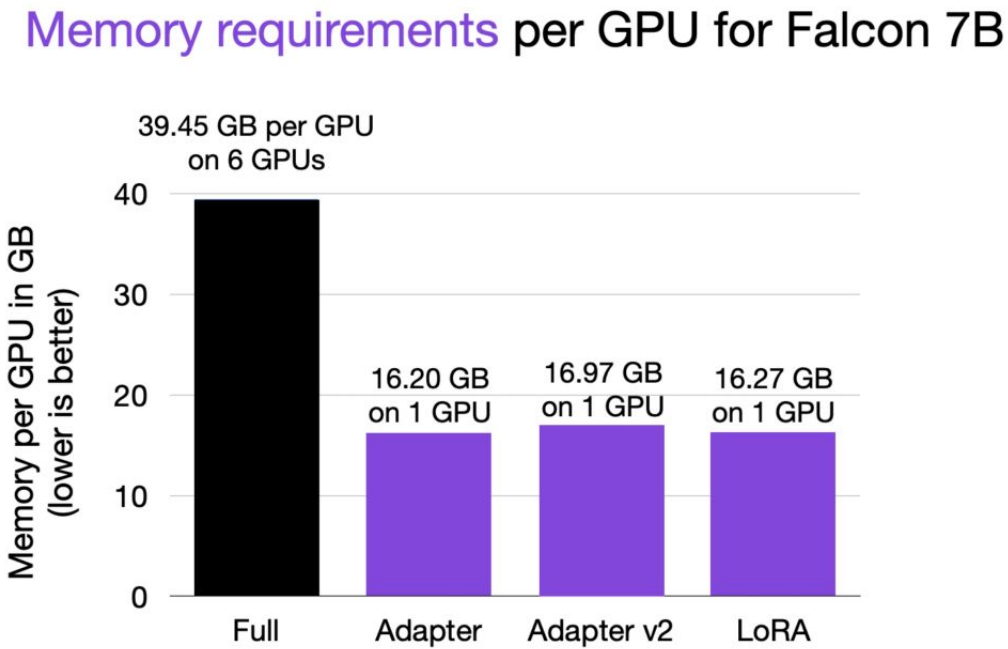
LoRA



Reference: Fine-tuning Falcon LLMs More Efficiently With LoRA and Adapters (link)

Fine-tuning Falcon 7b with LoRA

LoRA



Reference: Fine-tuning Falcon LLMs More Efficiently With LoRA and Adapters (link)

LoRA vs Prompt Tuning

LoRA

When to use LoRA?

- If you want to train the model on a much different task than what it has been trained on, LoRA is without a doubt a best strategy for tuning the model efficiently.

When to use Prompt Tuning?

- But if your task is more or less already understood by the model, but the challenge is to properly prompt the model, then you should use Prompt Tuning techniques. Prompt Tuning doesn’t modify many parameters in the model and mainly focuses on the passed prompt instead.

Parameter-Efficient Fine-Tuning

LLM w/o Parameter Tuning

Fine-tuning

Instruction Tuning

PEFT: Additive Methods

- Soft Prompting
- Adapters

PEFT: Low-Rank Adaptation

- LoRA
- **QLoRA**

Agenda.

Quantized LoRA (OLoRA)

QLoRA

Quantized LoRA (QLoRA):

- Proposed to further reduces memory usage during fine-tuning with LoRA.
- In the backpropagation process, QLoRA ***quantizes*** the pretrained weights to a ***4-bit precision*** and employs ***paged optimizers*** to manage memory spikes.
- LoRA with ***16-bit brain floating point*** precision:
  - Training time: 1.85 h
  - Memory used: 21.33 GB
- QLoRA with ***4-bit Normal Floats***:
  - Training time: 2.79 h
  - Memory used: 14.18 GB

https://magazine.sebastianraschka.com/p/practical-tips-for-fine-tuning-llms

Understanding Precision

QLoRA

- **Computation in Neural Networks:**
  involves mathematical operations on network weights and activations during the forward pass (predictions) and backward pass (weight updates in training).
- **Precision and Range in Neural Networks:**
  - Typical neural networks use 32-bit floating-point numbers for computations.
  - This choice balances precision (accurate number representation) and range (the range of representable numbers).
  - Using 32-bit floating-point numbers for all computations can be memory-intensive. For example, consider fine-tuning a Falcon 40B-Instruct LLM on ONE 40GB NVIDIA GPU.
  - Quantization is a technique employed to reduce the precision of numbers used in a model.
  - In 4-bit quantization, where a 4-bit integer can range from -8 to 7, weights and activations are compressed from 32-bit floating-point numbers to 4-bit integers. However, it is not possible to perform pure 4-bit training on LLM models.

Reference: Understanding 4bit Quantization: QLoRA explained (youtube)

Understanding Quantization

QLoRA

As an example, consider quantizing a 32-bit Floating Point tensor X into an Int8 tensor X with range [-127, 127]:

$$\mathbf{X}^{\text{Int8}} = \text{round}\left(\frac{127}{\text{absmax}(\mathbf{X}^{\text{FP32}})}\mathbf{X}^{\text{FP32}}\right) = \text{round}(c^{\text{FP32}} \cdot \mathbf{X}^{\text{FP32}}),$$

where c is the quantization constant.
Dequantization is the inverse process:

$$\text{dequant}(c^{\text{FP32}}, \mathbf{X}^{\text{Int8}}) = \frac{\mathbf{X}^{\text{Int8}}}{c^{\text{FP32}}} = \mathbf{X}^{\text{FP32}}$$

The drawback of this method is that when a large magnitude value, an outlier, exists in the input tensor, the quantization bins—specific bit combinations—are not effectively utilized, leading to few or no numbers being quantized in certain bins. To mitigate this outlier issue, a common approach involves dividing the input tensor into blocks. Each block is independently quantized, with its quantization constant c.

Reference: Understanding 4bit Quantization: QLoRA explained (youtube)

Understanding Quantization

QLoRA

Example:

- Let's 4-bit integers represent 16 levels evenly spaced between -1 and 1. These levels would be: [-1.0, -0.8667, -0.7333, -0.6, -0.4667, -0.3333, -0.2, -0.0667, 0.0667, 0.2, 0.3333, 0.4667, 0.6, 0.7333, 0.8667, 1.0]

- Consider a network weight whose 32-bit floating-point value is 0.5678. This weight value of 0.5678 is closest to 0.6, so we would quantize this weight to 0.6. In the 4-bit representation, 0.6 corresponds to the integer 13. We store the 4-bit integer 13 instead of the 32-bit floating-point number 0.5678.

- If we use this weight in a computation, we first dequantize it back (0.6) to the floating-point number. The dequantization error is 0.6 - 0.5678 = 0.0322. This is in fact, a 25% dequantization error.

Reference: Understanding 4bit Quantization: QLoRA explained (youtube)

Understanding Quantization

QLoRA

4-bit NormalFloat point quantization:

- **Normalization:**
  To ensure the weights are distributed around zero within a limited range, they are first normalized to have zero mean and unit variance.
- **Quantization:**
  The next step is to quantize the normalized weights into 4 bits. This process entails assigning the original high-precision weights to a reduced set of low-precision values. For NF4, the quantization levels are specifically selected to be evenly distributed within the range of the normalized weights.
- **Dequantization:**
  In both the forward pass and backpropagation, the quantized weights undergo dequantization to revert to full precision. This process involves mapping the 4-bit quantized values back to their original range. Although the dequantized weights are utilized in computations, they are stored in memory in their 4-bit quantized form.

Reference: Understanding 4bit Quantization: QLoRA explained (youtube)

QLoRA

QLoRA

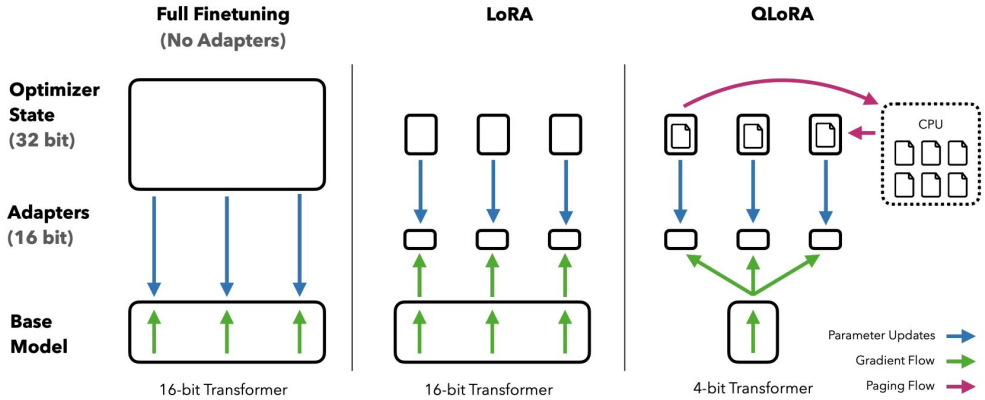


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

Reference: QLoRA: Efficient Fine-tuning of Quantized LLM (paper)

QLoRA

QLoRA

Quantized LoRA (QLoRA):

- **In forward pass:**
  - The LLM weights are stored in 4-bit quantized weights, and the injected LORA weights are stored in 32-bit.
  - all the 4-bit quantized weights are de-quantized into 32-bit weights. Since this is done during the computation, we do not need to store them in memory.
- **In backward pass:**
  - All the backward calculations are performed in 32-bits.
  - The Frozen weights are frozen regarding to memory and storage and not regarding the calculation.
  - Once the backward calculations are done, ONLY the 32-bit LORA weights are updated and stored in memory. Infact, from 100-weight tensor, you only adjust one weight, and all the other weights are kept frozen.

Reference: QLoRA: Efficient Fine-tuning of Quantized LLM (paper)

QLoRA

QLoRA

Quantized LoRA (QLoRA):

- **In summary:**
  - With the QLORA fine-tuning, the network learns the new task, given the dequantization error of the 4-bit quantized tensors.
  - Only the modified 32-bit LORA weights are stored in memory.
  - Furthermore, using NVIDIA *mixed precision training,* we balance the trade-off between accuracy and speed/memory usage.

Reference: QLoRA: Efficient Fine-tuning of Quantized LLM (paper)

QLoRA

QLoRA

Quantized LoRA (QLoRA):

- **In summary:**
  - After applying all the optimization steps, 4-bit QLORA performs almost equal to a bfloat16 model.

Table 4: Mean 5-shot MMLU test accuracy for LLaMA 7-65B models finetuned with adapters on Alpaca and FLAN v2 for different data types. Overall, NF4 with double quantization (DQ) matches BFloat16 performance, while FP4 is consistently one percentage point behind both.

	Mean 5-shot MMLU Accuracy								
LLaMA Size	7B		13B		33B		65B		Mean
Dataset	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	
BFloat16	38.4	45.6	47.2	50.6	57.7	60.5	61.8	62.5	53.0
Float4	37.2	44.0	47.3	50.0	55.9	58.5	61.3	63.3	52.2
NFloat4 + DQ	39.0	44.5	47.5	50.7	57.3	59.2	61.8	63.9	53.1

Reference: QLoRA: Efficient Fine-tuning of Quantized LLM (paper)

QLoRA

PEFT

Method	Type	Storage	Memory	Backprop	Inference overhead
Adapters (Houlsby et al., 2019)	A	yes	yes	no	Extra FFN
AdaMix (Wang et al., 2022)	A	yes	yes	no	Extra FFN
SparseAdapter (He et al., 2022b)	AS	yes	yes	no	Extra FFN
Cross-Attn tuning (Gheini et al., 2021)	S	yes	yes	no	No overhead
BitFit (Ben-Zaken et al., 2021)	S	yes	yes	no	No overhead
DiffPruning (Guo et al., 2020)	S	yes	no	no	No overhead
Fish-Mask (Sung et al., 2021)	S	yes	maybe[5]	no	No overhead
LT-SFT (Ansell et al., 2022)	S	yes	maybe[5]	no	No overhead
Prompt Tuning (Lester et al., 2021)	A	yes	yes	no	Extra input
Prefix-Tuning (Li and Liang, 2021)	A	yes	yes	no	Extra input
Spot (Vu et al., 2021)	A	yes	yes	no	Extra input
IPT (Qin et al., 2021)	A	yes	yes	no	Extra FFN and input
MAM Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN and input
Parallel Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN
Intrinsinc SAID (Aghajanyan et al., 2020)	R	no	no	no	No overhead
LoRa (Hu et al., 2022)	R	yes	yes	no	No overhead
UniPELT (Mao et al., 2021)	AR	yes	yes	no	Extra FFN and input
Compacter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
PHM Adapter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
KronA (Edalati et al., 2022)	R	yes	yes	no	No overhead
KronA$^{B}_{res}$ (Edalati et al., 2022)	AR	yes	yes	no	Extra linear layer
(IA)^3 (Liu et al., 2022)	A	yes	yes	no	Extra gating
Attention Fusion (Cao et al., 2022)	A	yes	yes	yes	Extra decoder
LeTS (Fu et al., 2021)	A	yes	yes	yes	Extra FFN
Ladder Side-Tuning (Sung et al., 2022)	A	yes	yes	yes	Extra decoder
FAR (Vucetic et al., 2022)	S	yes	maybe[6]	no	No overhead
S4-model (Chen et al., 2023)	ARS	yes	yes	no	Extra FFN and input

Reference: QLoRA: Efficient F

QLoRA

QLoRA: Quantized LoRA

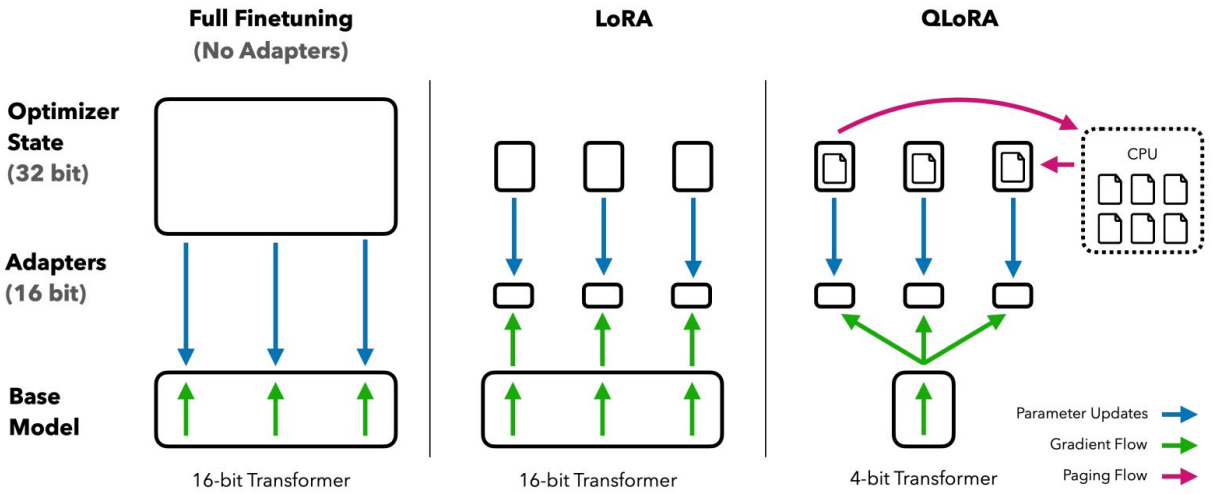


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

Method	Type	Storage	Memory	Backprop	Inference overhead
Adapters (Houlsby et al., 2019)	A	yes	yes	no	Extra FFN
AdaMix (Wang et al., 2022)	A	yes	yes	no	Extra FFN
SparseAdapter (He et al., 2022b)	AS	yes	yes	no	Extra FFN
Cross-Attn tuning (Gheini et al., 2021)	S	yes	yes	no	No overhead
BitFit (Ben-Zaken et al., 2021)	S	yes	yes	no	No overhead
DiffPruning (Guo et al., 2020)	S	yes	no	no	No overhead
Fish-Mask (Sung et al., 2021)	S	yes	maybe[5]	no	No overhead
LT-SFT (Ansell et al., 2022)	S	yes	maybe[5]	no	No overhead
Prompt Tuning (Lester et al., 2021)	A	yes	yes	no	Extra input
Prefix-Tuning (Li and Liang, 2021)	A	yes	yes	no	Extra input
Spot (Vu et al., 2021)	A	yes	yes	no	Extra input
IPT (Qin et al., 2021)	A	yes	yes	no	Extra FFN and input
MAM Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN and input
Parallel Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN
Intrinsinc SAID (Aghajanyan et al., 2020)	R	no	no	no	No overhead
LoRa (Hu et al., 2022)	R	yes	yes	no	No overhead
UniPELT (Mao et al., 2021)	AR	yes	yes	no	Extra FFN and input
Compacter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
PHM Adapter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
KronA (Edalati et al., 2022)	R	yes	yes	no	No overhead
KronA$^{B}_{res}$ (Edalati et al., 2022)	AR	yes	yes	no	Extra linear layer
(IA)^3 (Liu et al., 2022)	A	yes	yes	no	Extra gating
Attention Fusion (Cao et al., 2022)	A	yes	yes	yes	Extra decoder
LeTS (Fu et al., 2021)	A	yes	yes	yes	Extra FFN
Ladder Side-Tuning (Sung et al., 2022)	A	yes	yes	yes	Extra decoder
FAR (Vucetic et al., 2022)	S	yes	maybe[6]	no	No overhead
S4-model (Chen et al., 2023)	ARS	yes	yes	no	Extra FFN and input

Method	% Trainable parameters	% Changed parameters	Evaluated on <1B	<20B	>20B
Adapters (Houlsby et al., 2019)	0.1 - 6	0.1 - 6	yes	yes	yes
AdaMix (Wang et al., 2022)	0.1 - 0.2	0.1 - 0.2	yes	no	no
SparseAdapter (He et al., 2022b)	2.0	2.0	yes	no	no
BitFit (Ben-Zaken et al., 2021)	0.05 - 0.1	0.05 - 0.1	yes	yes	yes
DiffPruning (Guo et al., 2020)	200	**0.5**	yes	no	no
Fish-Mask (Sung et al., 2021)	0.01 - 0.5	0.01 - 0.5	yes	yes	no
Prompt Tuning (Lester et al., 2021)	0.1	0.1	yes	yes	yes
Prefix-Tuning (Li and Liang, 2021)	0.1 - 4.0	0.1 - 4.0	yes	yes	yes
IPT (Qin et al., 2021)	56.0	56.0	yes	no	no
MAM Adapter (He et al., 2022a)	0.5	0.5	yes	no	no
Parallel Adapter (He et al., 2022a)	0.5	0.5	yes	no	no
Intrinsinc SAID (Aghajanyan et al., 2020)	0.001 - 0.1	**∼0.1 or 100**	yes	yes	no
LoRa (Hu et al., 2022)	0.01 - 0.5	**∼0.5 or ∼30**	yes	yes	yes
UniPELT (Mao et al., 2021)	1.0	1.0	yes	no	no
Compacter (Karimi Mahabadi et al., 2021)	0.05-0.07	**∼0.07 or ∼0.1**	yes	yes	no
PHM Adapter (Karimi Mahabadi et al., 2021)	0.2	**∼0.2 or ∼1.0**	yes	no	no
KronA (Edalati et al., 2022)	0.07	**∼0.07 or ∼30.0**	yes	no	no
KronA$^{B}_{res}$ (Edalati et al., 2022)	0.07	**∼0.07 or ∼1.0**	yes	no	no
(IA)^3 (Liu et al., 2022)	0.02	0.02	no	yes	no
Ladder Side-Tuning(Sung et al., 2022)	7.5	7.5	yes	yes	no
FAR (Vucetic et al., 2022)	6.6-26.4	6.6-26.4	yes	no	no
S4-model (Chen et al., 2023)	0.5	**more than 0.5**	yes	yes	no